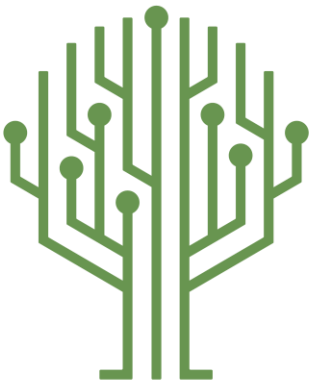


Green Pace Secure Development Policy

Contents

Overview

Software development at Green Pace requires consistent implementation of secure principles to all developed applications. Consistent approaches and methodologies must be maintained through all policies that are uniformly defined, implemented, governed, and maintained over time.

Purpose

This policy defines the core security principles; C/C++ coding standards; authorization, authentication, and standards; and data encryption standards. This article explains the differences between policy, standards, principles, and practices (guidelines and procedure): Understanding the Hierarchy of Principles, Policies, Standards, Procedures, and Guidelines.

Scope

This document applies to all staff that create, deploy, or support custom software at Green Pace.

Module Three Milestone

Ten Core Security Principles

Principles	Write a short paragraph explaining each of the 10 principles of security.
1. Validate Input Data	Validate any input that comes from untrusted data sources. Appropriate input validation can omit the majority of software vulnerabilities.
2. Heed Compiler Warnings	Compile all code using the highest warning levels supported by the compiler and address all warnings before deployment. Compiler warnings frequently identify unsafe constructs, type of mismatches, and undefined behavior that can introduce security vulnerabilities if left unresolved.
3. Architect and Design for Security Policies	Security requirements must be incorporated during the architecture and design phases of development. Designing systems around defined security policies ensures consistent enforcement of controls and reduces the risk of vulnerabilities caused by ad hoc or reactive security measures.
4. Keep It Simple	Software should be designed to be as simple as possible while meeting functional requirements. Reducing complexity limits the attack surface, lowers the likelihood of implementation errors, and improves the ability to analyze and secure the system.
5. Default Deny	Access to resources, functionality, and data must be denied by default unless explicitly permitted. This approach ensures that unintended access is prevented, and that only authorized actions are allowed within the system.
6. Adhere to the Principle of Least Privilege	Grant users, processes, and components with only the minimum privileges required to perform their intended functions. Limiting privileges reduces the potential impact of security failures and restricts an attacker's ability to escalate access.

Principles	Write a short paragraph explaining each of the 10 principles of security.
7. Sanitize Data Sent to Other Systems	All data passed to external systems must be validated and sanitized before use. Proper sanitization prevents untrusted data from being interpreted as executable commands or queries by downstream components.
8. Practice Defense in Depth	Implement multiple, independent layers of security controls to protect critical assets. Defense in depth ensures that the failure of a single control does not result in a complete compromise of the system
9. Use Effective Quality Assurance Techniques	Apply security focused quality assurance techniques such as code reviews, static analysis, and dynamic testing throughout the development lifecycle. These practices help identify vulnerabilities early and reduce the likelihood of security defects reaching production.
10. Adopt a Secure Coding Standard	Developers must follow an established secure coding standard when writing and maintaining software. Secure coding standards provide consistent guidance for avoiding common vulnerabilities and improving overall software security.

C/C++ Ten Coding Standards

Complete the coding standards portion of the template according to the Module Three milestone requirements. In Project One, follow the instructions to add a layer of security to the existing coding standards. Please start each standard on a new page, as they may take up more than one page. The first seven coding standards are labeled by category. The last three are blank, so you may choose three additional standards. Be sure to label them by category and give them a sequential number for that category. Add compliant and noncompliant sections as needed for each coding standard.

Coding Standard 1

Coding Standard	Label	Use safe, explicit types and conversions
Safe Explicit Types	STD-001-CPP	Type confusion and implicit conversions can cause truncation, sign issues, and undefined behavior. Using explicit width types and safe casts makes intent clear and reduces integer bugs.

Noncompliant Code

A negative int is silently converted to an unsigned size_t, producing a very large value that can trigger large allocations or crashes.

Noncompliant Code

```
int len = -1;
size_t n = len;          // converts -1 to a huge size_t
std::vector<char> buf(n); // may try to allocate enormous memory
```

Compliant Code

The code checks for invalid negative values before converting to size_t, preventing sign-conversion bugs and unsafe allocations.

```
int len = getLength();
if (len < 0) {
  throw std::runtime_error("length must be non-negative");
}
auto n = static_cast<size_t>(len);
std::vector<char> buf(n);
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 1: Validate Input Data – Values used in size calculations must be validated before conversion to prevent unsafe memory allocation.
Principle 2: Heed Compiler Warnings – Compiler warnings help detect unsafe implicit conversions and truncation risks.
Principle 10: Adopt a Secure Coding Standard – Enforcing explicit casting reduces systemic type confusion vulnerabilities.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Low	High	4

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x	C++ Security Rules	Detects unsafe casts, injection risks, memory errors.
Clang Tidy	LLVM	bugprone, cppcoreguidelines, hicpp	Flags narrowing conversions, suspicious casts, and unsafe type usage patterns.
Compiler Warnings (GCC/Clang/MSVC)	Current	Wall, Wextra, Wconversion, -Wsign-conversion (or MSVC /W4)	Detects implicit conversions, sign/size mismatches, and narrowing risks at compile time.

Tool	Version	Checker	Description Tool
Cppcheck	2.x text.	warning, style, portability	Detects suspicious conversions and type related defects that can lead to memory bugs.

Coding Standard 2

Coding Standard	Label	Validate numeric ranges and prevent integer overflow/underflow
Integer Overflow	STD-002-CPP	Integer values must be validated for expected ranges before they are used in calculations, memory sizing, indexing, or loop bounds. Integer overflow/underflow (including unsigned wraparound) and unsafe conversions can cause incorrect allocation sizes, out of bounds access, and other security relevant failures. Guidance in the CERT secure coding rules emphasizes preventing wraparound and ensuring conversions and operations do not produce misinterpreted or overflowed results.

Noncompliant Code

This code computes an allocation size using multiplication without checking for overflow. If the multiplication wraps, the program may allocate less memory than required and later write out of bounds.

```
#include #include

int* allocateInts(std::size_t count) {

std::size_t total = count *   sizeof(int); // overflow risk (wrap)

int* p = static_cast<int*>  (std::malloc(total));

return p; // caller may write count ints -> OOB

}
```

Compliant Code

This code validates that the multiplication cannot overflow before computing the allocation size. If the requested size is unsafe or allocation fails, it handles the error instead of continuing with an incorrect buffer size.

```
#include #include #include #include

 int* allocateInts(std::size_t count) {

 if (count > (std::numeric_limitsstd::size_t::max() / sizeof(int))) {
```

Compliant Code

```
throw std::overflow_error("allocation size overflow"); }

std::size_t total = count * sizeof(int);

int* p = static_cast<int*>(std::malloc(total));

if (!p) {

   throw std::bad_alloc();
}

return p;

}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 1: Validate Input Data – Numeric ranges must be validated before arithmetic operations are performed.
Principle 2: Fail Securely – When overflow is detected, the system must halt safely rather than continue in an unsafe state.
Principle 9: Use Effective Quality Assurance Techniques – Static analysis tools help detect overflow conditions early.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	High	Medium	High	5

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x text	C++ Security Rules	Detects arithmetic overflow and unsafe numeric conversions.
Clang Static Analyzer	Latest LLVM	core.UndefinedBinaryOperatorResult	Identifies potential overflow and undefined arithmetic behavior.
Cppcheck	2.x	warning, overflow	Detects possible integer wraparound and unsafe calculations.

Coding Standard 3

Coding Standard	Label	Guarantee that storage for strings has sufficient space for character data and the null terminator
Bounded String Handling	STD-003-CPP	Copying or reading character data into a destination buffer that is too small results in a buffer overflow. String related buffer overflows are common and can be exploited to execute arbitrary code or corrupt program state. Therefore, bounded storage must be enforced, and safer string abstractions should be used whenever possible.

Noncompliant Code

This code reads unbounded input into a fixed size character array. If the input exceeds the destination size, a buffer overflow can occur.

```
#include <iostream>

void f() {
    char buf[12];
    std::cin >> buf;    // Unbounded input; may overflow buf
}
```

Compliant Code

This code avoids writing into a fixed size character buffer by using `std::string`, which dynamically manages storage. This approach prevents classic string buffer overflows caused by oversized input.

```
#include <iostream>
#include <string>

void f() {
    std::string string_one, string_two;
    std::cin >> string_one >> string_two;    // Uses dynamically sized   storage
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 1: Validate Input Data – Input size must be validated against buffer capacity before storage.
Principle 8: Practice Defense in Depth – Safer abstractions such as std::string provide layered protection.
Principle 10: Adopt a Secure Coding Standard – Enforcing bounded string handling prevents overflow vulnerabilities.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	High	Medium	High	5

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x	C++ Security Rules	Detects unsafe string handling and buffer overflows.
Clang Static Analyzer	Latest LLVM	core.StackAddressEscape	Identifies buffer misuse and memory corruption risks.
AddressSanitizer	Compiler runtime	ASan	Detects runtime buffer overflows and memory violations.

Coding Standard 4

Coding Standard	Label	Prevent SQL injection by avoiding dynamic query construction
SQL Injection	STD-004-CPP	Constructing SQL statements by concatenating untrusted input allows attackers to alter query logic. SQL injection vulnerabilities can lead to unauthorized data access, data modification, or complete system compromise and must be prevented through parameterized queries or prepared statements.

Noncompliant Code

This code builds an SQL query by directly concatenating user input, allowing malicious input to change the query structure.

```
std::string query =
    "SELECT * FROM users WHERE username = '" + user + "'";
executeQuery(query)
```

Compliant Code

This code uses a parameterized query, ensuring that user input is treated strictly as data and not executable SQL.

```
PreparedStatement stmt("SELECT * FROM users WHERE username = ?");
stmt.bind(1, user);
stmt.execute();
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 1: Validate Input Data – User input must be treated strictly as data and not executable commands.

Principle 3: Minimize Attack Surface – Avoiding dynamic SQL reduces exposure to injection attacks.

Principle 7: Sanitize Data Sent to Other Systems – Parameterized queries sanitize outbound database operations.

Principle 10: Adopt a Secure Coding Standard – Enforcing prepared statements prevents injection vulnerabilities system-wide.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Low	High	5

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x text	Injection Rules	Detects unsafe dynamic query construction.
OWASP ZAP	2.x	SQL Injection Scanner	Performs dynamic testing for injection vulnerabilities.
Snyk / Dependency Scanner	Current	DB Security Checks	Detects vulnerable DB drivers and libraries.

Coding Standard 5

Coding Standard	Label	Manage dynamic memory safely and prevent use after free errors
Memory Safety	STD-005-CPP	Improper memory management can result in use after free, double free, and memory leaks. These vulnerabilities are frequently exploited to corrupt memory or execute arbitrary code, making safe ownership and lifetime management essential.

Noncompliant Code

Manage dynamic memory safely and prevent use after free errors

`int* p = new int(10);`
`delete p;`
`*p = 5;  // use-after-free`

Compliant Code

This code uses automatic memory management to ensure memory is released safely and not reused incorrectly.

`#include <memory>`
 	
`auto p = std::make_unique<int>(10);`
`*p = 5;`

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 2: Fail Securely – Improper memory operations must not result in exploitable system states.
Principle 8: Practice Defense in Depth – Smart pointers and runtime sanitizers provide layered protection.
Principle 9: Use Effective Quality Assurance Techniques – Static and dynamic analysis detect memory corruption.
Principle 10: Adopt a Secure Coding Standard – Safe memory management must be enforced consistently.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Critical	High	High	Critical	5

Automation

Tool	Version	Checker	Description Tool
Valgrind	Latest	Memcheck	Detects use-after-free and memory leaks.
AddressSanitizer	Compiler runtime	ASan	Detects heap and stack memory corruption.
Clang Static Analyzer	Latest	core.NullDereference, cplusplus.NewDelete	Detects improper memory usage.

Coding Standard 6

Coding Standard	Label	Use assertions only to validate internal program assumptions
Assertions	STD-006-CPP	Assertions are intended to detect programming errors during development and must not be used to enforce runtime security checks. Assertions may be disabled in production builds, making them unsuitable for validating untrusted input.

Noncompliant Code

This code relies on an assertion to validate user input, which may be disabled at runtime.

```
assert(userAge >= 18);
grantAccess();
```

Compliant Code

This code performs an explicit runtime check to enforce the security requirement regardless of build configuration.

```
if (userAge < 18) {
    denyAccess();
    return;
 }
 grantAccess();
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 2: Fail Securely – Security controls must not rely on assertions that may be disabled in production builds.
Principle 9: Use Effective Quality Assurance Techniques – Testing ensures assertions are not misused as runtime security controls.
Principle 10: Adopt a Secure Coding Standard – Runtime validation must be enforced explicitly and consistently.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Medium	Medium	Medium	3

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x	Reliability Rules	Detects misuse of assertions for runtime validation.
Clang Tidy	LLVM	Bugprone assert side effect	Identifies unsafe assertion patterns.

Coding Standard 7

Coding Standard	Label	Handle exceptions securely and avoid suppressing failures
Exceptions	STD-007-CPP	Improper exception handling can mask errors and allow programs to continue execution in an unsafe state. Exceptions should be handled deliberately, logged appropriately, and either resolved or safely propagated.

Noncompliant Code

This code catches all exceptions without handling them, allowing execution to continue in an unknown state.

```
try {
    riskyOperation();
} catch (...) {
    // ignored
}
```

Compliant Code

This code catches standard exceptions, logs the error, and fails safely.

```
try {
    riskyOperation();
} catch (const std::exception& e) {
    logError(e.what());
    throw;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 2: Fail Securely – Exceptions must not expose sensitive data or leave the system unstable.
Principle 8: Practice Defense in Depth – Secure error handling adds another layer of protection.
Principle 9: Use Effective Quality Assurance Techniques – Testing ensures proper propagation and containment of exceptions.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Medium	Medium	Medium	4

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x	Reliability Rules	Detects empty catch blocks and swallowed exceptions.
Clang Tidy	LLVM	bugprone exception escape	Flags unsafe exception handling patterns.

Coding Standard 8

Coding Standard	Label	Validate pointers before dereferencing
Pointer Validation	STD-008-CPP	Validate pointers before dereferencing to prevent null pointer dereference and undefined behavior.

Noncompliant Code

This code dereferences a pointer without validating it.

```
void f(int* p) {
    *p = 10;
}
```

Compliant Code

This code validates the pointer before use, preventing null dereference.

```
void f(int* p) {
    if (p != nullptr) {
        *p = 10;
    }
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 1: Validate Input Data – Pointers derived from external input must be validated before dereferencing.
Principle 2: Fail Securely – Null or invalid pointers must be handled safely.
Principle 9: Use Effective Quality Assurance Techniques – Static analysis tools detect potential null dereference vulnerabilities.
Principle 10: Adopt a Secure Coding Standard – Pointer validation must be enforced consistently.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Low	High	4

Automation

Tool	Version	Checker	Description Tool
Clang Static Analyzer	Latest	core.NullDereference	Detects potential null pointer dereference.
SonarQube	10.x	nullPointer	Detects null pointer usage.
Cppcheck	2.x	nullPointer	Detects null pointer usage.

Coding Standard 9

Coding Standard	Label	Log security relevant errors without exposing sensitive data
Logging	STD-009-CPP	Logging sensitive information such as passwords or secrets creates additional attack vectors. Logs should capture sufficient diagnostic information without exposing confidential data.

Noncompliant Code

This code logs sensitive authentication information.

```
log("Login failed: user=" + user + " password=" + password);
```

Compliant Code

This code logs the failure without exposing sensitive credentials.

```
log("Login failed for user=" + user);
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 1: Validate Input Data – Sensitive information must not be written to logs.
Principle 3: Minimize Attack Surface – Logs containing secrets increase exposure risk.
Principle 8: Practice Defense in Depth – Removing credentials from logs adds another layer of protection.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Low	High	4

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x	Security Rules	Detects hardcoded credentials and sensitive logging.
GitLeaks	Latest	Secret Scanner	Detects exposed secrets in logs and code.

Coding Standard 10

Coding Standard	Label	Configure systems to operate securely by default
Secure Defaults	STD-010-CPP	Insecure default configurations expose systems to unintended access. Secure defaults ensure that explicit action is required to enable potentially dangerous functionality.

Noncompliant Code

This code enables guest access by default.

```
bool allowGuest = true;
```

Compliant Code

This code disables guest access unless explicitly enabled.

```
bool allowGuest = false;
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):
Principle 5: Default Deny Access – Systems must deny access by default unless explicitly authorized.
Principle 6: Least Privilege – Default configurations must restrict unnecessary permissions.
Principle 8: Practice Defense in Depth – Secure defaults reduce exposure even if other controls fail.
Principle 10: Adopt a Secure Coding Standard – Secure configuration must be enforced consistently.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Low	High	4

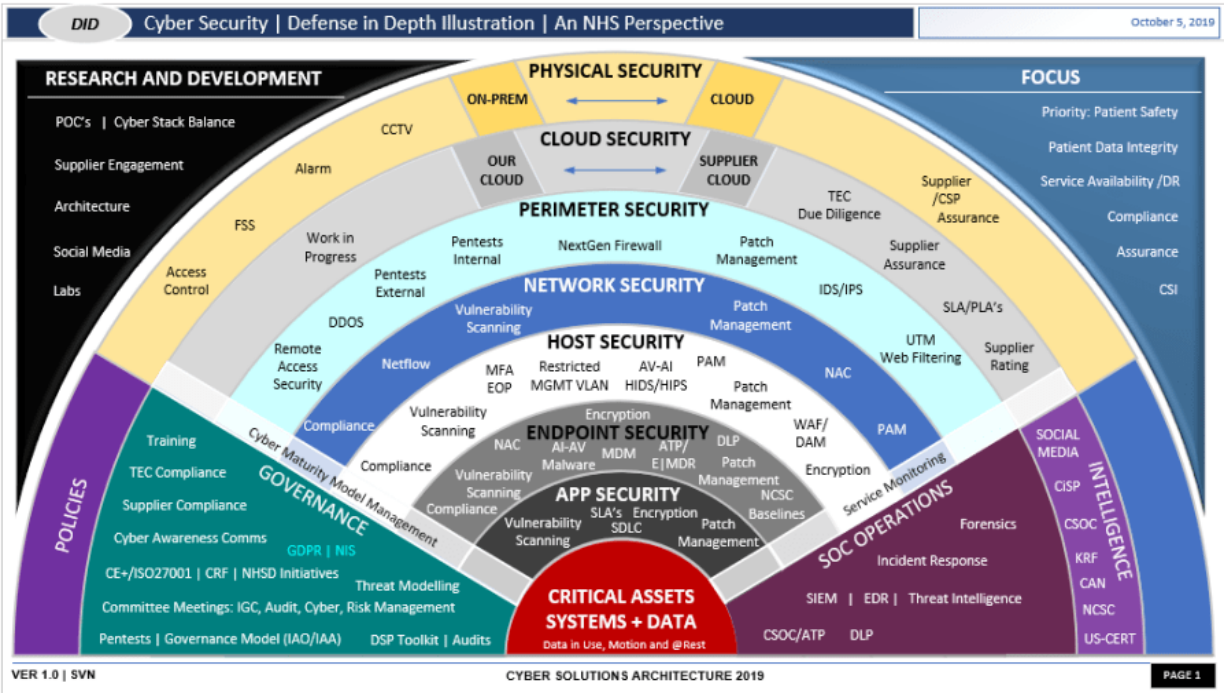
Automation

Tool	Version	Checker	Description Tool
SonarQube	10.x	Security Hotspots	Flags insecure default configurations.

Tool	Version	Checker	Description Tool
Static Code Review	Current	Config Rules	Detects insecure default settings in code.

Defense-in-Depth Illustration

This illustration provides a visual representation of the defense-in-depth best practice of layered security.



Project One

There are seven steps outlined below that align with the elements you will be graded on in the accompanying rubric. When you complete these steps, you will have finished the security policy.

Revise the C/C++ Standards

You completed one of these tables for each of your standards in the Module Three milestone. In Project One, add revisions to improve the explanation and examples as needed. Add rows to accommodate additional examples of compliant and noncompliant code. Coding standards begin on the security policy.

Risk Assessment

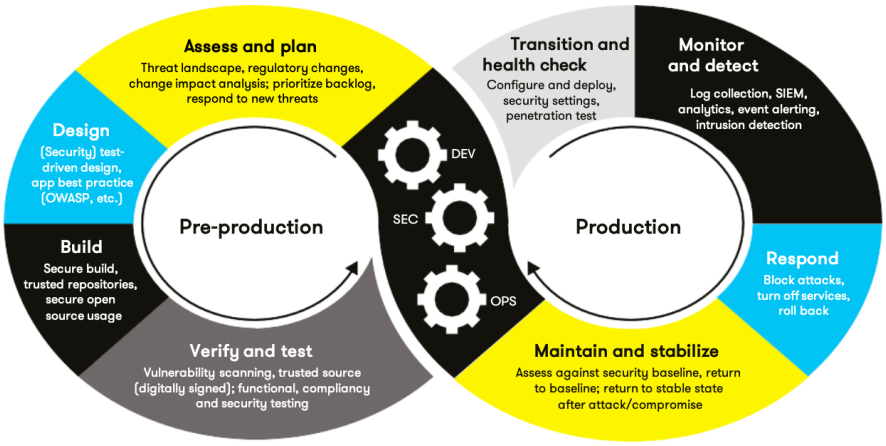
Complete this section on the coding standards tables. Enter high, medium, or low for each of the headers, then rate it overall using a scale from 1 to 5, 5 being the greatest threat. You will address each of the seven policy standards. Fill in the columns of severity, likelihood, remediation cost, priority, and level using the values provided in the appendix.

Automated Detection

Complete this section of each table on the coding standards to show the tools that may be used to detect issues. Provide the tool name, version, checker, and description. List one or more tools that can automatically detect this issue and its version number, name of the rule or check (preferably with link), and any relevant comments or description, if any. This table ties to a specific C++ coding standard.

Automation

Provide a written explanation using the image provided.



Automation will be used for the enforcement of and compliance to the standards defined in this policy. Green Pace already has a well established DevOps process and infrastructure. Define guidance on where and how to modify the existing DevOps process to automate enforcement of the standards in this policy. Use the DevSecOps diagram and provide an explanation using that diagram as context.

Summary of Risk Assessments

Consolidate all risk assessments into one table including both coding and systems standards, ordered by standard number.

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-001-CPP	High	Medium	Low	High	4
STD-002-CPP	High	High	Medium	High	5
STD-003-CPP	High	High	Medium	High	5
STD-004-CPP	High	Medium	Low	High	5
STD-005-CPP	Critical	High	High	Critical	5
STD-006-CPP	Medium	Medium	Low	Medium	3
STD-007-CPP	Medium	Medium	Medium	Medium	4
STD-008-CPP	High	Medium	Low	High	4
STD-009-CPP	High	Medium	Low	High	4
STD-010-CPP	High	Medium	Low	High	4

Create Policies for Encryption and Triple A

Include all three types of encryption (in flight, at rest, and in use) and each of the three elements of the Triple-A framework using the tables provided**.**

a. Explain each type of encryption, how it is used, and why and when the policy applies.
b. Explain each type of Triple-A framework strategy, how it is used, and why and when the policy applies.

Write policies for each and explain what it is, how it should be applied in practice, and why it should be used.

a. Encryption	**Explain what it is and how and why the policy applies.**
Encryption at rest	This policy applies whenever sensitive data is persisted beyond active memory, including backups, archived logs, and replicated databases. The purpose is to preserve confidentiality in the event of physical theft, unauthorized system access, or storage compromise.
Encryption in flight	All data transmitted across networks must use TLS 1.2 or higher. Certificates must be validated and managed using secure certificate authorities. This policy applies to all internal and external network communications. Encryption in transit protects against man in the middle attacks and unauthorized interception of sensitive data.
Encryption in use	Sensitive data processed in memory must be protected where possible using secure enclaves or memory protection mechanisms. Memory must be cleared after use to prevent data leakage. This policy applies whenever sensitive data is processed in memory. Clearing memory after use reduces the risk of memory scraping attacks and residual data exposure.

b. Triple-A Framework*	**Explain what it is and how and why the policy applies.**
Authentication	All users must authenticate using secure credentials. Multifactor authentication is required for privileged accounts. Passwords must be hashed using Argon2 or PBKDF2. This policy applies to all user accounts, service accounts, and administrative access.
Authorization	Access must follow the principle of least privilege. Role based access control (RBAC) must be enforced. Default deny applies to all system components. This policy applies whenever access to data, systems, or functionality is requested. Enforcing least privilege limits damage if an account is compromised.
Accounting	All authentication attempts, privilege changes, database updates, and file access must be logged. Logs must be retained and protected against tampering. Regular log reviews

b. Triple-A Framework*	Explain what it is and how and why the policy applies.
	are required. This policy supports forensic investigation, regulatory compliance, and detection of insider threats.

*****Use this checklist for the Triple A to be sure you include these elements in your policy:

- User logins
- Changes to the database
- Addition of new users
- User level of access
- Files accessed by users

Map the Principles

Map the principles to each of the standards, and provide a justification for the connection between the two. In the Module Three milestone, you added definitions for each of the 10 principles provided. Now it's time to connect the standards to principles to show how they are supported by principles. You may have more than one principle for each standard, and the principles may be used more than once. Principles are numbered 1 through 10. You will list the number or numbers that apply to each standard, then explain how each of these principles supports the standard. This exercise demonstrates that you have based your security policy on widely accepted principles. Linking principles to standards is a best practice.

NOTE: Green Pace has already successfully implemented the following:

- Operating system logs
- Firewall logs
- Anti-malware logs

The only item you must complete beyond this point is the Policy Version History table.

Audit Controls and Management

Every software development effort must be able to provide evidence of compliance for each software deployed into any Green Pace managed environment.

Evidence will include the following:

- Code compliance to standards
- Well-documented access-control strategies, with sampled evidence of compliance
- Well-documented data-control standards defining the expected security posture of data at rest, in flight, and in use
- Historical evidence of sustained practice (emails, logs, audits, meeting notes)

Enforcement

The office of the chief information security officer (OCISO) will enforce awareness and compliance of this policy, producing reports for the risk management committee (RMC) to review monthly. Every system deployed in any environment operated by Green Pace is expected to be in compliance with this policy at all times.

Staff members, consultants, or employees found in violation of this policy will be subject to disciplinary action, up to and including termination.

Exceptions Process

Any exception to the standards in this policy must be requested in writing with the following information:

- Business or technical rationale
- Risk impact analysis
- Risk mitigation analysis
- Plan to come into compliance
- Date for when the plan to come into compliance will be completed

Approval for any exception must be granted by chief information officer (CIO) and the chief information security officer (CISO) or their appointed delegates of officer level.

Exceptions will remain on file with the office of the CISO, which will administer and govern compliance.

Distribution

This policy is to be distributed to all Green Pace IT staff annually. All IT staff will need to certify acceptance and awareness of this policy annually.

Policy Change Control

This policy will be automatically reviewed annually, no later than 365 days from the last revision date. Further, it will be reviewed in response to regulatory or compliance changes, and on demand as determined by the OCISO.

Policy Version History

Version	Date	Description	Edited By	Approved By
1.0	02/14/2026	Completed Project One security policy revisions including risk assessment, automation	Asia Mayfield	CISO

Appendix A Lookups

Approved C/C++ Language Acronyms

Language	Acronym
C++	CPP
C	CLG
Java	JAV

References

Software Engineering Institute, Carnegie Mellon University. (2015–2023).

SEI CERT C++ Coding Standard.

https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=88046682

Software Engineering Institute, Carnegie Mellon University. (2018).

Top 10 Secure Coding Practices.

https://wiki.sei.cmu.edu/confluence/spaces/seccode/pages/88042842/Top+10+Secure+Coding+Practices

Varne, A. (2021).

Secure coding guidelines.

https://varneanand-88.medium.com/secure-coding-guidelines-126890bca67a

OWASP Foundation. (2023).

OWASP Top 10 Web Application Security Risks.

https://owasp.org/www-project-top-ten/